MEDIACREST
TRAINING COLLEGE

Mediacrest Training College

Nairobi, Kenya

Advanced Data Analytics & Machine Learning

8-Week Advanced Training Program

Comprehensive Session Notes

Week 7 · Day 1

Unsupervised Learning: K-Means Clustering & Principal Component Analysis (PCA)

Course Trainer: Victor Wandera Lumumba, MSc.
Role: Statistician, Data Analyst & ML Expert

Session Date: Sept 21st, 2026
Mode: Hybrid – Online / Physical (EAT)

"Unsupervised learning is the art of letting the data speak for itself. Without the crutch of labeled outcomes, we must rely on mathematical rigor, distance metrics, and variance maximization to uncover the hidden structures that govern our world."

About This Document

These comprehensive notes accompany **Week 7, Day 1** of the **Advanced Data Analytics & Machine Learning** program at Mediacrest Training College. This session pivots from supervised learning to **Unsupervised Learning**, providing deep mathematical and practical foundations for two cornerstone algorithms: **K-Means Clustering** and **Principal Component Analysis (PCA)**.

Building on real-world datasets (the Palmer Penguins dataset and a Cardiovascular Disease Risk dataset), these notes bridge the gap between abstract mathematical theory (objective functions, eigen-decomposition, distance metrics) and applied pattern recognition in biological and clinical contexts.

Course at a Glance

Duration	8 Weeks (20+ intensive sessions)
Core Tools	Python (pandas, NumPy, scikit-learn, matplotlib, seaborn)
Focus	Mathematical foundations, algorithmic assumptions, cluster evaluation, dimensionality reduction
Capstone	Production-ready ML system with diagnostics, explainability & policy recommendations
Context	Biological Taxonomy, Healthcare, Clinical Risk Prediction, and Epidemiological Modeling

How to Use These Notes

- **Master the Math:** The **Mathematical Foundation** boxes contain derivations essential for understanding *why* algorithms behave the way they do.
- **Follow along in code:** Every listing is runnable. Open the companion Jupyter Notebook and execute each cell as you read.
- **Contextualize:** **Real-World Context** boxes connect abstract math to local realities, whether ecological conservation or Kenyan healthcare constraints.

Contents

About This Document 1

1 Foundations of Unsupervised Learning & K-Means Theory 3
- 1.1 The Unsupervised Learning Paradigm . . . 3
- 1.2 The K-Means Clustering Algorithm . . . 3
- 1.3 Lloyd's Algorithm: The Iterative Process . . . 3
- 1.4 Strengths and Limitations . . . 4

2 Case Study I: Clustering the Palmer Penguins 4
- 2.1 Dataset Overview . . . 4
- 2.2 Data Preprocessing Pipeline . . . 4

3 Determining the Optimal Number of Clusters (k) 5
- 3.1 The Elbow Method (WCSS) . . . 5
- 3.2 Silhouette Score . . . 6
- 3.3 Davies-Bouldin Index (DBI) . . . 6

4 K-Means Implementation & Model Evaluation 7
- 4.1 Fitting the Final Model ($k = 3$) . . . 7
- 4.2 Comprehensive Model Evaluation . . . 8
- 4.3 Crosstab Analysis: Clusters vs. True Species . . . 8

5 Principal Component Analysis (PCA): Mathematical Foundations 9
- 5.1 The Curse of Dimensionality & The Need for PCA . . . 9
- 5.2 Explained Variance Ratio . . . 9

6 Case Study II: PCA on Cardiovascular Disease (CVD) Risk 9
- 6.1 Dataset & Preprocessing . . . 9
- 6.2 Determining Optimal Components . . . 10
- 6.3 Component Interpretation (Loadings) . . . 10

7 Downstream Modeling: Raw vs. PCA-Transformed Features 11
- 7.1 Performance Comparison . . . 11

8 Summary, Best Practices & Policy Recommendations 12
- 8.1 The Reusable K-Means Pipeline . . . 12
- 8.2 Algorithm Selection Guide . . . 13

9 Glossary of Key Mathematical Terms 13

10 References & Further Reading 13

1 Foundations of Unsupervised Learning & K-Means Theory

1.1 The Unsupervised Learning Paradigm

Unlike supervised learning, which predicts a known target variable y given features X, unsupervised learning deals with unlabeled data $\mathcal{D} = \{x_i\}_{i=1}^{N}$. The fundamental goal is to discover the underlying structure, distribution, or hidden patterns within the data without explicit guidance. Common tasks include clustering (grouping similar instances), dimensionality reduction (compressing information), and anomaly detection.

1.2 The K-Means Clustering Algorithm

K-Means is one of the simplest and most widely used partitional clustering algorithms. It operates under three critical geometric assumptions:

1. Clusters are **spherical** (isotropic).
2. Clusters are **well-separated**.
3. Clusters are of **roughly equal size** and density.

The algorithm seeks to partition the dataset into k distinct, non-overlapping clusters by minimizing the **Within-Cluster Sum of Squares (WCSS)**, also known as inertia.

The K-Means Objective Function

Let $C = \{C_1, C_2, \ldots, C_k\}$ be the set of k clusters, and μ_j be the centroid (mean vector) of cluster C_j. The objective is to find the assignment of points to clusters and the centroid locations that minimize the WCSS:

$$J(C, \mu) = \sum_{j=1}^{k} \sum_{x_i \in C_j} ||x_i - \mu_j||^2$$

where $|| \cdot ||^2$ denotes the squared Euclidean distance. This is a non-convex optimization problem, meaning K-Means guarantees convergence to a *local* minimum, not necessarily the global minimum.

1.3 Lloyd's Algorithm: The Iterative Process

The standard K-Means algorithm (Lloyd's algorithm) proceeds in an iterative two-step process:

1. **Assignment Step:** Assign each data point x_i to the cluster C_j whose centroid μ_j is closest in Euclidean distance:

$$C_j = \{x_i : ||x_i - \mu_j||^2 \leq ||x_i - \mu_m||^2 \quad \forall m \neq j\}$$

2. **Update Step:** Recalculate the centroid of each cluster as the mean of all points currently assigned to it:

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

These steps repeat until convergence, defined as the point where cluster assignments no longer change (or the change in WCSS falls below a tiny tolerance ϵ).

K-Means++ Initialization

Random initialization of centroids can lead to poor local minima. **K-Means++** mitigates this by strategically choosing initial centroids:

1. Choose the first centroid uniformly at random from the data points.
2. For each subsequent centroid, compute the distance $D(x)$ from each point to the nearest already chosen centroid.
3. Choose the next centroid with probability proportional to $D(x)^2$.
4. Repeat until k centroids are chosen.

This ensures initial centroids are spread out, drastically improving convergence speed and final cluster quality.

1.4 Strengths and Limitations

- **Strengths:** Computationally efficient ($O(N \cdot k \cdot d \cdot i)$, where i is iterations), highly scalable, and easy to interpret.
- **Limitations:** Requires pre-specifying k; sensitive to outliers (since the mean is easily skewed); struggles with non-convex or elongated cluster shapes (e.g., moons or concentric circles); assumes equal cluster variance.

2 Case Study I: Clustering the Palmer Penguins

2.1 Dataset Overview

The Palmer Penguins dataset is a modern alternative to the Iris dataset, containing morphological measurements of penguins across three species on islands in the Palmer Archipelago, Antarctica.

- **Observations:** 344 rows.
- **Features:** 8 columns (`species`, `island`, `bill_length_mm`, `bill_depth_mm`, `flipper_length_mm`, `body_mass_g`, `sex`, `year`).

Real-World Context: Ecological & Biological Modeling

While this dataset is ecological, the techniques mirror clinical phenotyping. Just as we group penguins by morphological traits to identify species, clinicians group patients by biomarkers (e.g., BMI, blood pressure, cholesterol) to identify disease endotypes. The mathematical machinery is identical.

2.2 Data Preprocessing Pipeline

K-Means is a distance-based algorithm. Therefore, rigorous preprocessing is mandatory.

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, LabelEncoder

# 1. Load Data
df = pd.read_csv('penguins.csv')

# 2. Handle Missing Values
# The dataset has minimal missing values: 2 in numerical features, 11 in 'sex'.
# We drop incomplete rows to ensure a perfectly clean distance matrix.
df_clean = df.dropna()
print(f"Rows after removing missing values: {len(df_clean)}") # Output: 333

# 3. Feature Selection
# K-Means requires numerical data. We isolate the morphological measurements.
numerical_features = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', '
    body_mass_g']
X = df_clean[numerical_features].values

# 4. Target Extraction (for external validation later)
y_true = df_clean['species'].values
le = LabelEncoder()
y_encoded = le.fit_transform(y_true)
```

Listing 1: Data Loading and Cleaning

Mandatory Feature Scaling

Features like `body_mass_g` (values $\approx 3000 - 5000$) will completely dominate the Euclidean distance calculation over `bill_depth_mm` (values $\approx 15 - 20$). **Standardization (Z-score scaling)** is strictly mandatory to ensure each feature contributes equally to the distance metric.

```python
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Verify scaling: Mean should be ~0, Std should be ~1
print(f"Mean after scaling: {X_scaled.mean(axis=0).round(4)}")
print(f"Std after scaling: {X_scaled.std(axis=0).round(4)}")
```

Listing 2: Feature Standardization

3 Determining the Optimal Number of Clusters (k)

Since K-Means requires k as an input, we must use quantitative heuristics to determine the optimal number of clusters.

3.1 The Elbow Method (WCSS)

We fit K-Means for a range of k values (e.g., 2 to 10) and plot the inertia. The "elbow" is the point where the rate of decrease sharply slows, indicating diminishing returns for adding more clusters.

```
inertias = []
K_range = range(2, 11)
for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    inertias.append(kmeans.inertia_)
# Plotting code omitted for brevity, but generates the classic elbow curve.
```

Listing 3: Elbow Method Implementation

3.2 Silhouette Score

The Silhouette Score provides a more rigorous, quantitative assessment of cluster quality by measuring both *cohesion* (how close points are to their own cluster) and *separation* (how far points are from the nearest neighboring cluster).

Silhouette Score Derivation

For a single data point i:

- $a(i)$: Mean distance between i and all other points in the *same* cluster (cohesion).
- $b(i)$: Mean distance between i and all points in the *nearest neighboring* cluster (separation).

The silhouette coefficient $s(i)$ for point i is:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

The overall Silhouette Score is the mean of $s(i)$ over all data points. **Interpretation:** Ranges from -1 to 1. Close to 1 indicates well-defined clusters; ≈ 0 indicates overlapping clusters; negative values indicate misassigned points.

3.3 Davies-Bouldin Index (DBI)

The DBI evaluates the average similarity between each cluster and its most similar cluster, based on cluster scatter (standard deviation) and centroid distance.

Davies-Bouldin Index

$$DB = \frac{1}{k} \sum_{i=1}^{k} \max_{j \neq i} \left(\frac{\sigma_i + \sigma_j}{d(\mu_i, \mu_j)} \right)$$

where σ_i is the average distance of points in cluster i to its centroid μ_i, and $d(\mu_i, \mu_j)$ is the distance between centroids. **Interpretation: Lower is better**. A score of 0 represents ideal, perfectly separated clusters.

```
from sklearn.metrics import silhouette_score, davies_bouldin_score

silhouette_scores = []
db_scores = []
```

```
5
6 for k in K_range:
7     kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
8     labels = kmeans.fit_predict(X_scaled)
9     silhouette_scores.append(silhouette_score(X_scaled, labels))
10    db_scores.append(davies_bouldin_score(X_scaled, labels))
11
12 print(f"Best k by Silhouette Score: {K_range[silhouette_scores.index(max(
       silhouette_scores))]}")
13 print(f"Best k by Davies-Bouldin Score: {K_range[db_scores.index(min(db_scores))
       ]}")
14 # Output: Best k by Silhouette Score: 2
15 # Output: Best k by Davies-Bouldin Score: 2
```

Listing 4: Silhouette and Davies-Bouldin Evaluation

The Domain Knowledge Override

Mathematically, both Silhouette and Davies-Bouldin strongly suggest $k = 2$, implying the data naturally forms two major morphological groupings (likely separating the large Gentoo from the smaller Adelie/Chinstrap). However, **domain knowledge** dictates that there are exactly **3** penguin species. Therefore, we proceed with $k = 3$ to align the unsupervised model with biological ground truth, accepting a slight mathematical trade-off for real-world interpretability.

4 K-Means Implementation & Model Evaluation

4.1 Fitting the Final Model ($k = 3$)

```
k = 3
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10, max_iter=300)
kmeans.fit(X_scaled)

# Extract labels and add to dataframe
cluster_labels = kmeans.labels_
df_clean['cluster'] = cluster_labels

# Inverse transform cluster centers to original scale for interpretation
cluster_centers_scaled = kmeans.cluster_centers_
cluster_centers = scaler.inverse_transform(cluster_centers_scaled)
print(pd.DataFrame(cluster_centers, columns=numerical_features))
```

Listing 5: Final K-Means Execution

Interpretation of Cluster Centers (Original Scale):

- **Cluster 0:** Smaller body size (Bill Length $\approx$ 38.3mm, Mass $\approx$ 3594g). Likely Adelie.
- **Cluster 1:** Largest penguins (Bill Length $\approx$ 47.6mm, Mass $\approx$ 5092g, Flipper $\approx$ 217mm). Likely Gentoo.
- **Cluster 2:** Intermediate size, but deepest bill (Bill Depth $\approx$ 18.7mm, Mass $\approx$ 3898g). Likely Chinstrap.

4.2 Comprehensive Model Evaluation

We evaluate the clustering using both **internal** metrics (no ground truth needed) and **external** metrics (comparing to true species labels).

```
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score

# Internal Metrics
sil = silhouette_score(X_scaled, cluster_labels)
db = davies_bouldin_score(X_scaled, cluster_labels)
inertia = kmeans.inertia_

# External Metrics (vs true encoded species)
ari = adjusted_rand_score(y_encoded, cluster_labels)
nmi = normalized_mutual_info_score(y_encoded, cluster_labels)

print(f"Silhouette: {sil:.4f} | Davies-Bouldin: {db:.4f} | Inertia: {inertia:.2f}")
print(f"Adjusted Rand Index (ARI): {ari:.4f} | NMI: {nmi:.4f}")
```

Listing 6: Internal and External Evaluation Metrics

Results:

- **Silhouette:** 0.4462 (Moderate separation, some expected biological overlap).
- **Davies-Bouldin:** 0.9420 (Acceptable compactness).
- **Adjusted Rand Index (ARI):** 0.7994. An ARI near 0.8 indicates strong agreement between the unsupervised clusters and the true biological species, adjusted for chance.
- **Normalized Mutual Information (NMI):** 0.7899. Confirms a high level of shared information between the clustering output and true labels.

4.3 Crosstab Analysis: Clusters vs. True Species

```
comparison = pd.DataFrame({'True Species': y_true, 'Cluster Label': cluster_labels})
print(pd.crosstab(comparison['True Species'], comparison['Cluster Label'],
                  rownames=['Species'], colnames=['Cluster']))
```

Listing 7: Crosstabulation of Predictions vs. Truth

Output Interpretation:

- **Cluster 1** perfectly captures all 119 **Gentoo** penguins (0 misclassifications). This species is morphologically distinct.
- **Cluster 0** mainly represents **Adelie** (124 correct), with 22 misclassified into Cluster 2.
- **Cluster 2** mainly represents **Chinstrap** (63 correct), with 5 misclassified into Cluster 0.

The overlap between Adelie and Chinstrap is expected, as they are morphologically similar and sometimes hybridize in the wild. Despite this, the model has successfully captured the underlying biological structure.

5 Principal Component Analysis (PCA): Mathematical Foundations

5.1 The Curse of Dimensionality & The Need for PCA

As the number of features d increases, the volume of the feature space grows exponentially, causing data to become sparse. Distance metrics lose their discriminative power, and models overfit. **Principal Component Analysis (PCA)** is a linear dimensionality reduction technique that transforms a large set of correlated variables into a smaller set of uncorrelated variables (Principal Components, PCs) while retaining as much variance (information) as possible.

The Mathematical Mechanics of PCA

Given a standardized data matrix $X \in \mathbb{R}^{N \times d}$ (mean-centered, variance=1):

1. **Covariance Matrix:** Compute the $d \times d$ covariance matrix $C = \frac{1}{N-1} X^T X$.
2. **Eigen-decomposition:** Solve the characteristic equation $Cv = \lambda v$, where λ are the eigenvalues and v are the corresponding eigenvectors.
3. **Ranking:** Sort the eigenvalue-eigenvector pairs in descending order of λ. The eigenvector with the largest λ is the **First Principal Component (PC1)**, capturing the direction of maximum variance in the data.
4. **Orthogonality:** Subsequent components (PC2, PC3, etc.) are constrained to be orthogonal (uncorrelated) to all preceding components.
5. **Projection:** Transform the original data into the new subspace: $X_{pca} = XW_k$, where W_k is the matrix of the top k eigenvectors.

5.2 Explained Variance Ratio

The proportion of total variance explained by the j-th principal component is:

$$\text{Explained Variance Ratio}_j = \frac{\lambda_j}{\sum_{m=1}^{d} \lambda_m}$$

We typically select k components such that the **cumulative explained variance** exceeds a threshold (e.g., 90% or 95%).

6 Case Study II: PCA on Cardiovascular Disease (CVD) Risk

6.1 Dataset & Preprocessing

We apply PCA to a clinical dataset predicting CVD Risk, featuring 8 variables: `Systolic_BP`, `Diastolic_BP`, `Total_Cholesterol`, `HDL_Cholesterol`, `Fasting_Glucose`, `CRP_Inflammation`, `Age`, and `LDL_Cholesterol`.

Kenyan Clinical Context

In Kenya, Non-Communicable Diseases (NCDs) like hypertension and diabetes are rising rapidly. These biomarkers are highly multicollinear (e.g., high LDL often correlates with high Total Cholesterol). PCA can distill these 8 redundant tests into 2-3 composite "risk scores," simplifying clinical decision-making for Community Health Promoters (CHPs) in resource-constrained county facilities.

```
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split

# 1. Train/Test Split (BEFORE scaling to prevent data leakage)
X = df[features].values
y = df['CVD_Risk'].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
    random_state=42, stratify=y)

# 2. Standardization (Mandatory for PCA)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 3. Fit PCA on ALL components to evaluate variance
pca_full = PCA(random_state=42)
X_train_pca_full = pca_full.fit_transform(X_train_scaled)

explained_var = pca_full.explained_variance_ratio_
cumulative_var = np.cumsum(explained_var)
```

Listing 8: PCA Preprocessing and Fitting

6.2 Determining Optimal Components

```
threshold = 0.90
n_components_opt = np.argmax(cumulative_var >= threshold) + 1
print(f"Explained Variance per Component: {np.round(explained_var, 3)}")
print(f"Cumulative Variance: {np.round(cumulative_var, 3)}")
print(f"Optimal Components for {threshold*100}% Variance: {n_components_opt}")
```

Listing 9: Variance Thresholding

Output:

- Explained Variance: $[0.662, 0.167, 0.124, 0.018, \ldots]$
- Cumulative Variance: $[0.662, 0.829, \mathbf{0.952}, 0.970, \ldots]$
- **Optimal Components: 3** (Captures 95.2% of the total variance, exceeding the 90% threshold).

6.3 Component Interpretation (Loadings)

To understand what each PC represents, we examine the **loadings** (the weights of original features in each eigenvector).

```python
pca_opt = PCA(n_components=3, random_state=42)
X_train_pca = pca_opt.fit_transform(X_train_scaled)

loadings = pd.DataFrame(
    pca_opt.components_.T,
    columns=['PC1', 'PC2', 'PC3'],
    index=features
)
print(loadings.round(3))
```

Listing 10: Extracting and Interpreting Loadings

Clinical Interpretation of Loadings:

- **PC1 (Cardiometabolic Risk Axis):** High positive loadings for Total Cholesterol (0.421), LDL (0.416), Systolic BP (0.403), and Fasting Glucose (0.387). High *negative* loading for HDL (-0.396). This PC perfectly encapsulates the "metabolic syndrome" profile.
- **PC2 (Inflammation Axis):** Dominated by CRP Inflammation (0.812), with minor contributions from Glucose and HDL. This captures systemic inflammatory variation independent of pure lipid profiles.
- **PC3 (Age Axis):** Almost entirely driven by Age (0.995). This is a pure demographic dimension, largely orthogonal to the biochemical markers.

7 Downstream Modeling: Raw vs. PCA-Transformed Features

Does dimensionality reduction improve predictive modeling? We compare a Logistic Regression model trained on raw scaled features versus one trained on the 3 PCA components.

```python
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score, classification_report

# Model A: Raw Scaled Features
model_raw = LogisticRegression(max_iter=1000, random_state=42)
model_raw.fit(X_train_scaled, y_train)
y_pred_raw = model_raw.predict(X_test_scaled)
y_prob_raw = model_raw.predict_proba(X_test_scaled)[:, 1]

# Model B: PCA-Transformed Features
model_pca = LogisticRegression(max_iter=1000, random_state=42)
model_pca.fit(X_train_pca, y_train)
X_test_pca = pca_opt.transform(X_test_scaled)
y_pred_pca = model_pca.predict(X_test_pca)
y_prob_pca = model_pca.predict_proba(X_test_pca)[:, 1]
```

Listing 11: Comparative Model Training

7.1 Performance Comparison

- **Raw Model:** Test Accuracy = 0.8400, ROC-AUC = 0.9252.
- **PCA Model (3 Components):** Test Accuracy = **0.8533**, ROC-AUC = 0.9200.

Why Did PCA Improve Accuracy?

Although ROC-AUC dipped marginally, overall accuracy **improved**. By reducing 8 highly correlated features down to 3 orthogonal components, PCA eliminated multicollinearity and noise. This acts as an implicit regularizer, preventing the Logistic Regression model from overfitting to redundant signals (e.g., counting both LDL and Total Cholesterol as separate, compounding evidence). The model generalizes better to the unseen test set.

Furthermore, the classification report for the PCA model shows an improved **Recall for High Risk (0.88 vs 0.85)**. In clinical settings, identifying high-risk patients (minimizing False Negatives) is often the primary objective, making the PCA-enhanced model highly valuable.

8 Summary, Best Practices & Policy Recommendations

8.1 The Reusable K-Means Pipeline

To ensure reproducibility in production environments, encapsulate the entire workflow into a single function.

```
def perform_kmeans_clustering(data_path, n_clusters=3, random_state=42):
    df = pd.read_csv(data_path).dropna()
    features = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', '
    body_mass_g']
    X = df[features].values
    y_true = df['species']

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    kmeans = KMeans(n_clusters=n_clusters, random_state=random_state, n_init=10)
    labels = kmeans.fit_predict(X_scaled)

    # Comprehensive Metrics
    metrics = {
        "Silhouette Score": silhouette_score(X_scaled, labels),
        "Davies-Bouldin Score": davies_bouldin_score(X_scaled, labels),
        "Adjusted Rand Index": adjusted_rand_score(y_true, labels),
        "Inertia": kmeans.inertia_
    }

    return {
        'dataframe': df.assign(cluster=labels),
        'model': kmeans,
        'scaler': scaler,
        'cluster_centers': scaler.inverse_transform(kmeans.cluster_centers_),
        'metrics': metrics
    }
```

Listing 12: Production-Ready Clustering Function

8.2 Algorithm Selection Guide

- **K-Means:** Use for large, numerical datasets where clusters are expected to be spherical and roughly equal in size. *Always scale first.*
- **PCA:** Use as a preprocessing step when features are highly correlated (multicollinearity), when visualization of high-dimensional data is needed, or to denoise data before supervised modeling.

Capstone Preparation: The Algorithmic Audit

Imagine you are deploying the PCA + Logistic Regression model to a Kenyan county hospital. The hospital administrator asks: *"Your model flagged this patient as high risk, but I can't read 'PC1'. What does that mean for the patient?"*

1. Explain how you would map the high PC1 score back to the original loadings (e.g., "The patient's score is driven primarily by elevated LDL and Fasting Glucose").
2. Draft a 2-sentence response to the administrator that translates the mathematical abstraction into actionable clinical advice without losing statistical integrity.

9 Glossary of Key Mathematical Terms

Within-Cluster Sum of Squares (WCSS) The objective function minimized by K-Means, representing the sum of squared Euclidean distances between each point and its assigned cluster centroid.

Silhouette Coefficient A metric ranging from -1 to 1 that measures how similar a data point is to its own cluster compared to other clusters, balancing cohesion and separation.

Adjusted Rand Index (ARI) An external clustering evaluation metric that measures the similarity between two clusterings (e.g., predicted vs. true), adjusted for the probability of chance agreement.

Eigenvalue / Eigenvector In PCA, the eigenvector defines the direction of a principal component (the new axis), while the eigenvalue represents the amount of variance in the data captured along that direction.

Explained Variance Ratio The proportion of the dataset's total variance that is attributable to a specific principal component.

Loadings The coefficients of the linear combination of original variables that make up a principal component. High absolute loadings indicate strong influence.

10 References & Further Reading

- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning.* Springer. (Chapter 9 for K-Means and EM; Chapter 12 for PCA).

- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer. (Definitive mathematical text on clustering and dimensionality reduction).
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning with Applications in Python*. Springer.
- Horst, A. M., Hill, A. P., & Gorman, K. B. (2020). *palmerpenguins: Palmer Archipelago (Antarctica) penguin data*. R package version 0.1.0.
- Mediacrest Training College. (2026). *Advanced Data Analytics & Machine Learning Curriculum*.

Mediacrest Training College – Nairobi, Kenya

Advanced Data Analytics & Machine Learning · Week 7, Day 1 Comprehensive Notes

Trainer: Victor Wandera Lumumba, MSc. · September 2026
